



南開大學
Nankai University

计算机学院
并行程序设计报告

GPU 编程实验

姓名：王毅豪

学号：2412509

专业：计算机科学与技术

2026 年 6 月 20 日

目录

1 实验目标与内容	2
1.1 基础部分：HIP 编程课程（7 分）	2
1.2 进阶部分：高斯消去 CUDA 并行化（2 分）	2
2 实验环境	2
2.1 AMD AUP 云端平台（基础部分）	2
2.2 NVIDIA GPU 平台（进阶部分）	3
3 HIP 与 CUDA 编程模型概述	3
3.1 GPU 线程层次结构	3
3.2 Wavefront 与 Warp	3
3.3 GPU 内存层次	3
3.4 内存合并访问（Memory Coalescing）	4
4 基础部分：HIP 编程课程实验	4
4.1 Lab 1: Vector Addition ——GPU 编程入门	4
4.1.1 实验内容	4
4.1.2 核函数代码	4
4.1.3 实验结果与分析	4
4.2 Lab 2: Matrix Transpose ——内存合并访问	5
4.2.1 实验内容	5
4.2.2 Shared Memory Tiling 优化	5
4.2.3 实验结果	5
4.3 Lab 3: Histogram ——原子操作与竞争	5
4.3.1 实验内容	5
4.3.2 两级归约优化	5
4.3.3 竞争实验	6
4.4 Lab 4: Parallel Reduction ——树形归约与 Warp Shuffle	6
4.4.1 实验内容	6
4.4.2 优化递进	6
4.5 Lab 5: Monte Carlo Integration ——复合并行模式	6
4.6 Lab 6: K-Means Clustering ——多核函数综合	6
5 进阶部分：高斯消去的 CUDA 并行化	7
5.1 算法设计	7
5.2 核心代码	7
5.3 调试经验	8
5.4 性能测试结果与加速比	8
5.5 加速比分析	8
6 总结	8

A Lab 1: Vector Addition 填空答案	11
B Lab 2: Matrix Transpose 填空答案	13
C Lab 3: Histogram 填空答案	14
D Lab 4: Parallel Reduction 填空答案	15
E Lab 5: Monte Carlo Integration 填空答案	16
F Lab 6: K-Means Clustering 填空答案	17

1 实验目标与内容

本次 GPU 编程实验包含基础部分和进阶部分两个模块，共计 9 分（满分 10 分）。其中基础部分为 AMD AUP 平台的 HIP 编程课程学习（7 分），进阶部分为高斯消去算法的 CUDA 并行化实现（2 分）。

1.1 基础部分：HIP 编程课程（7 分）

基础部分要求在 AMD AUP (AMD University Program) 云端学习平台上完成 HIP (Heterogeneous Compute Interface for Portability) 编程课程的全部六个实验。HIP 是 AMD 开发的 GPU 编程框架，其 API 与 NVIDIA CUDA 高度相似——HIP 提供了一组 C++ 扩展和运行时库，允许开发者用统一的代码库同时编译到 AMD 和 NVIDIA 两种 GPU 平台上。对于熟悉 CUDA 的开发者而言，HIP 的学习曲线非常平缓。

六个实验按难度递进，系统覆盖了 GPU 编程的核心概念：

1. **Lab 1 - Vector Addition:** GPU 编程入门，理解线程层次结构 (Grid/Block/Thread) 和 wavefront 概念
2. **Lab 2 - Matrix Transpose:** 二维索引与内存合并访问 (Memory Coalescing), Shared Memory Tiling 优化
3. **Lab 3 - Histogram:** 原子操作 (Atomic Operations) 与竞争 (Contention) 分析，两级归约优化
4. **Lab 4 - Parallel Reduction:** 树形归约、Warp Shuffle、多元素/线程优化策略
5. **Lab 5 - Monte Carlo Integration:** Embarrassingly Parallel + Reduction 复合模式
6. **Lab 6 - K-Means Clustering:** 多核函数协作的迭代算法，综合前五课的所有概念

1.2 进阶部分：高斯消去 CUDA 并行化（2 分）

进阶部分选取高斯消去选题的 GPU 编程方向。使用 CUDA C++ 编程模型，在 NVIDIA Tesla T4 GPU 上实现普通高斯消去算法的并行化。算法设计与前四次实验 (Lab2 SIMD、Lab3 多线程、Lab4 MPI) 一脉相承：将消去循环的归一化步骤和消去步骤分别拆分成两个 GPU 核函数 (Kernel)，每个线程独立处理矩阵的一列或一行，利用 GPU 海量线程的并行计算能力实现加速。

2 实验环境

2.1 AMD AUP 云端平台（基础部分）

基础部分六个实验全部在 AMD AUP Learning Cloud 提供的云端环境中完成。该平台基于 JupyterLab 交互界面，预装了完整的 ROCm 运行时和 HIP 编译器 (hipcc)，无需本地配置即可直接运行 GPU 代码。

- **硬件配置：** AMD Radeon 8060S Graphics (RDNA 3.5 架构, gfx1151 核心)，40 个 Compute Units (HIP 运行时识别为 20 个双 CU 单元)，默认 wave32 模式，支持 wave32 与 wave64 双模式。集成 LPDDR5x 共享内存，峰值带宽约 256 GB/s。

- **软件栈：**ROCm 开源运行时，HIP 编译器（hipcc），JupyterLab 交互式 Notebook 环境。
- **学习材料：**六个 Jupyter Notebook，每个包含理论学习、代码填空、实验运行和性能分析四个环节。

2.2 NVIDIA GPU 平台（进阶部分）

进阶部分的 CUDA 高斯消去实验在 Google Colab 平台上完成，该平台提供免费的云端 GPU 计算资源。

- **硬件配置：**NVIDIA Tesla T4 GPU，基于 Turing 架构（SM75），拥有 40 个 SM（Streaming Multiprocessor），共计 5120 个 CUDA Core，核心频率 1590 MHz，配备 15.6 GB GDDR6 显存。
- **软件栈：**CUDA Toolkit（nvcc 编译器，-O2 优化），C++17 标准。

3 HIP 与 CUDA 编程模型概述

3.1 GPU 线程层次结构

GPU 编程的核心抽象是将一个大规模数据并行任务分解为海量线程同时执行。线程的组织遵循三级层次结构：

- **Grid（网格）：**一次核函数启动所创建的全部线程的集合。一个 Grid 可以包含多个 Block。
- **Block（线程块）：**一组可以相互协作的线程，共享同一块 Shared Memory（AMD 上称为 LDS——Local Data Share），并可通过 `__syncthreads()` 进行块内同步。同一 Block 内的所有线程被调度到同一个 Compute Unit / SM 上执行。
- **Thread（线程）：**最小执行单元。每个线程通过内置变量 `blockIdx`、`blockDim`、`threadIdx` 计算自己的全局索引，从而确定要处理的数据元素。

3.2 Wavefront 与 Warp

在 AMD GPU 上，一个 Block 内的线程被进一步分组为 **Wavefront**（NVIDIA 术语为 Warp）。RDNA 3.5 架构默认使用 wave32 模式（32 个线程为一组），也支持 wave64 模式（通过 `-mwavefrontsize64` 编译选项）。Wavefront 是 GPU 的硬件调度和执行单元——一个 Wavefront 内的所有 32 个线程同时执行同一条指令（SIMT 模型：Single Instruction, Multiple Thread）。如果同一 Wavefront 内的线程发生分支发散（Branch Divergence），两边的路径都会串行执行，导致性能下降。

Wavefront Size 对性能的影响：Block Size 必须是 Wavefront Size 的整数倍，否则会产生子波前（Sub-Wavefront）浪费。例如在 wave32 模式下，block size = 16 会浪费 50% 的 SIMD 通道。实验数据表明，子波前 block（BS=8: 0.0694 ms）比波前对齐 block（BS=128: 0.0184 ms）慢了 3.8 倍。

3.3 GPU 内存层次

GPU 拥有多层次的内存系统，各级之间的延迟和带宽差异显著：

表 1: GPU 内存层次对比 (AMD RDNA 3.5 / NVIDIA Turing)

内存类型	AMD 术语	典型延迟	容量
Register	VGPR	~0 cycles	每线程 256 个
Shared Memory	LDS	~20 cycles	每 CU 64-128 KB
L2 Cache	L2 Cache	~100 cycles	2-4 MB
Global Memory	HBM/LPDDR	~400 cycles	数 GB

Shared Memory (LDS) 是 GPU 上最重要的可编程缓存——延迟仅为 Global Memory 的 1/20。大多数性能优化技巧 (Tiling、两级归约、私有化) 都围绕如何将数据从 Global Memory 搬移到 Shared Memory 上来展开。

3.4 内存合并访问 (Memory Coalescing)

当同一 Wavefront 内的连续线程访问连续的内存地址时, GPU 的硬件可以将这些访问合并为一次大粒度的内存事务 (如 128 字节), 从而最大化带宽利用率。相反, 如果连续线程访问跨步地址 (如 Matrix Transpose 的写操作), 每个线程需要单独发起一次内存事务, 带宽利用率可骤降至 1/32。Lab 2 的 Naive Transpose 在大矩阵 (1001×2001) 上实测有效带宽仅约几十 GB/s, 远低于 256 GB/s 的硬件峰值。

4 基础部分：HIP 编程课程实验

本节详细描述六个 HIP 实验的学习内容、关键代码和实验结果。每个实验的填空答案详见附录 A-F。

4.1 Lab 1: Vector Addition ——GPU 编程入门

4.1.1 实验内容

向量加法是 GPU 编程的“Hello World”。任务描述为: 给定两个长度为 N 的浮点向量 A 和 B, 计算 $C[i] = A[i] + B[i]$ 。串行实现直接一个 for 循环即可解决。GPU 版本中, 每个线程负责计算一个元素: 线程通过 `blockIdx.x`、`blockDim.x` 和 `threadIdx.x` 计算全局索引 `idx`, 若 `idx < N` 则执行 $C[idx] = A[idx] + B[idx]$ 。

4.1.2 核函数代码

Vector Addition Kernel

```

1  __global__ void vector_add(const float* A, const float* B, float* C, int N) {
2      int idx = blockIdx.x * blockDim.x + threadIdx.x;
3      if (idx < N) {
4          C[idx] = A[idx] + B[idx];
5      }
6  }
```

4.1.3 实验结果与分析

实验测试了 $N=2^{20}$ (约 100 万元素), 每个 block size 跑 10 次取均值。主要发现:

- **最优 Block Size:** BS=512 获得最低平均执行时间 0.0179 ms, BS=1024 次之 (0.0273 ms), BS=16 (子波前) 最慢 (0.0555 ms)
- **子波前惩罚:** BS=8 (0.0694 ms) 和 BS=16 (0.0367 ms) 分别是 BS=128 (0.0184 ms) 的 3.8 倍和 2.0 倍
- **Memory-Bound 特性:** Block Size > 32 (波前对齐) 后性能差异不大, 因为 Vector Add 的计算/访存比极低 (每个线程 1 次加法 $\times$ 3 次内存访问), 瓶颈在内存带宽而非计算
- **Occupancy 分析:** 由于 $\text{max threads/CU} = 2048$ 和 $\text{max blocks/CU} = 64$ 的硬件限制, BS 32-256 的 block size 的 Active WF/CU 均为 64, occupancy 相同

4.2 Lab 2: Matrix Transpose ——内存合并访问

4.2.1 实验内容

矩阵转置看似简单的 $A^T[j][i] = A[i][j]$ 在 GPU 上却揭示了内存访问模式的深刻影响。以行优先 (Row-Major) 存储的矩阵, Naive Transpose 在读操作上是合并的 (连续线程读连续地址), 但在写操作上变成了跨步访问 (stride = rows), 同一 Wavefront 的 32 个线程需要 32 次独立的内存事务。

4.2.2 Shared Memory Tiling 优化

优化的核心思路是用 Shared Memory 做中转: 以 32×32 的 tile 为单位, 先合并读入 tile $\rightarrow$ `__syncthreads` 同步 $\rightarrow$ 合并写出。tile 声明为 [32][33] (多加一列) 是为了避免 Bank Conflict——Shared Memory 有 32 个 bank, 32×32 的同一列所有行映射到同一个 bank, 加一列错开地址。

4.2.3 实验结果

小矩阵 (16×16 、 1×1024) 由 kernel launch overhead 主导: 虽然只有几百个元素, 执行时间与中等规模测试接近 (0.0079 ms)。大矩阵 (1001×2001 , 约 200 万元素) 中跨步写入的问题充分暴露: 有效带宽远低于 256 GB/s 峰值, tiled 版本有望弥补这一差距。

4.3 Lab 3: Histogram ——原子操作与竞争

4.3.1 实验内容

直方图问题是原子操作的经典应用: 统计输入数组中每个值出现的次数。当多个线程同时尝试更新同一个 bin 时, 产生 Race Condition。`atomicAdd` 保证读-改-写操作的原子性, 但代价是所有冲突的访问被串行化执行。

4.3.2 两级归约优化

优化策略: 先私有化到 Shared Memory (每个 block 维护一份本地 histogram), 用本地 atomics 累加 (LDS 延迟 ~ 20 cycles), sync 后再用 global atomics 合并到全局结果 (~ 400 cycles)。这一优化将 global atomics 从 $O(N)$ 降至 $O(k \times \text{gridDim})$ (k 为 bin 数), 实际性能提升极为显著: $N=5000$ 万时, Optimized GPU (1.79 ms) 比 Naive GPU (360.59 ms) 快 201.8 倍, 比 Serial CPU (24.58 ms) 快 13.8 倍。

4.3.3 竞争实验

三种数据分布的测试验证了竞争对性能的影响：均匀分布 (0.1871 ms) → 低竞争；高斯分布 (0.1865 ms) → 中等竞争（集中在中心 bin）；全同值 (0.2706 ms) → 最高竞争，比均匀分布慢 45%。由于共享内存私有化的隔离作用，竞争影响被限制在 block 内部，global memory 不受影响。

4.4 Lab 4: Parallel Reduction ——树形归约与 Warp Shuffle

4.4.1 实验内容

归约 (Reduction) 是将 N 个元素合并为一个结果的算子 (sum、max、min 等)。Naive 方案让所有线程 atomicAdd 到同一个全局变量， $O(N)$ 的串行化使 GPU 优势荡然无存。

4.4.2 优化递进

实验对比了四种策略的单步优化效果：

- **Tree (Shared Memory)**: 每 block 内部先在 LDS 上做 $\log_2(\text{blockDim})$ 步两两归约 (stride 从 128 依次减半至 1)，仅最后一步做 global atomicAdd。将 global atomics 从 N 降为 blockDim。
- **Tree + Warp Shuffle**: 最后 $\log_2(\text{warpSize})$ 步用 `__shfl_down` 替代 shared memory。Warp 内线程是 SIMT 锁步执行的，无需 `__syncthreads`，消除了 bank conflict 和同步开销。
- **Multi-Element + Tree + Shuffle**: 每个线程先通过 grid-stride loop 处理多个元素，增大算术强度。 $N=100$ 万时比 Naive 快 2.41 倍。

关键教训：Occupancy $\neq$ Performance。 所有 block size 的 occupancy 相同（受限于 $\text{max threads}/\text{CU}=2048$ ），但算法效率差异导致性能差距超过 2 倍。

4.5 Lab 5: Monte Carlo Integration ——复合并行模式

Monte Carlo 积分的 GPU 实现是“Embarrassingly Parallel + Reduction”的复合体。积分近似值 $= (b - a)/n \times \sum y_i$ ，样本计算完全独立 (embarrassingly parallel)，但求和需要 reduction。

实验结果验证了小规模任务中 I/O overhead 的主导地位： N 从 10 万增加到 1000 万 (100 倍)，执行时间仅增加 10.4 倍 (非 100 倍)，因为大测试摊薄了 kernel launch 和 I/O 开销。在大规模下，逐步接近线性扩展，但受 atomicAdd 竞争限制。精度方面，float (7 位有效数字) 在 $N > 10^7$ 时出现截断误差，double (15 位) 是更安全的选择。

4.6 Lab 6: K-Means Clustering ——多核函数综合

K-Means 是综合实验，融合了前五课的所有概念。算法包含三个迭代执行的核函数：

1. **find_nearest_centroids (分配步)**: 每个线程处理一个数据点，计算到 k 个质心的欧氏距离，选择最近的。 $O(N \times k)$ 的距离计算，embarrassingly parallel。
2. **recalculate_centroid (更新步)**: 用两级原子归约 (LDS $\rightarrow$ global) 累加每个簇的所有点坐标， $O(N)$ 共享内存 atomics。

3. **finalize_and_compare** (收敛检测): 单 block 执行, 将累加值除以点数得到新质心, 检查位移是否 $< 10^{-8}$ 。

实验在 $N=50,000$ 、 $k=50$ 、 $\text{max_iter}=100$ 的配置下总耗时 0.118 秒。每次迭代的主开销来自 assignment step (2.5M 次距离计算), update step 因共享内存优化耗时仅为 1/5。

5 进阶部分：高斯消去的 CUDA 并行化

5.1 算法设计

普通高斯消去算法包含三层嵌套循环。在第 k 轮消去中: (1) 除法 (归一化) —— 将第 k 行除以主元 $A[k][k]$; (2) 消去 —— 将下方所有行 $i = k + 1 \dots N - 1$ 的第 k 列消为 0。两个步骤被拆分为两个 CUDA 核函数, 分别利用 GPU 的不同并行维度。

除法核函数 (division_kernel): 每个线程负责一列。 $N - k - 1$ 个列元素由 $N - k - 1$ 个线程并行处理 (线程不足时自动补齐为 block size 整数倍)。由于多个 block 同时读写 $A[k][k]$, 为了避免 block 间的竞争条件, 将“设置对角线为 1.0”的逻辑拆分到独立的单线程核函数 `set_diag_kernel` 中执行。

消去核函数 (eliminate_kernel): 采用一行一个 block 的映射方式。第 k 轮消去有 $N - k - 1$ 个待消去行, 对应启动 $N - k - 1$ 个 block。Block 内的所有线程 (256 个) 通过 grid-stride loop 覆盖该行的全部列 ($N - k - 1$ 列), 利用 `__syncthreads` 同步后仅 thread 0 清除第 k 列元素为 0。

5.2 核心代码

除法核函数与对角线设置

```
1  __global__ void division_kernel(float* A, int k, int N) {
2      int tid = blockIdx.x * blockDim.x + threadIdx.x;
3      int col = k + 1 + tid;
4      if (col < N) {
5          float diag = A[k * N + k];
6          if (fabs(diag) > 1e-30f) A[k * N + col] /= diag;
7      }
8  }
9  __global__ void set_diag_kernel(float* A, int k, int N) {
10     A[k * N + k] = 1.0f; // 单线程, 无竞争
11 }
```

消去核函数

```
1  __global__ void eliminate_kernel(float* A, int k, int N) {
2      int row = k + 1 + blockIdx.x; // 每个 block 负责一行
3      if (row >= N) return;
4      float factor = A[row * N + k];
5      for (int c = k + 1 + threadIdx.x; c < N; c += blockDim.x)
6          A[row * N + c] -= factor * A[k * N + c];
7      __syncthreads();
8      if (threadIdx.x == 0) A[row * N + k] = 0.0f;
9  }
```

主循环每轮依次启动除法核函数、对角线设置核函数和消去核函数, 通过 `cudaDeviceSynchronize()` 确保所有核函数执行完毕后进入下一轮。 N 轮消去共启动 $3N$ 次核函数, 每次 `cudaDeviceSynchronize` 引入约 5-10 μs 的 CPU-GPU 同步延迟。

5.3 调试经验

初次实现遭遇了两个关键 bug：（1）矩阵初始化——最初的”上三角 + 行组合”方案在 $N=2048$ 时因累加溢出产生 inf，改用严格对角占优矩阵 ($|A_{ii}| > \sum_{j \neq i} |A_{ij}|$) 后解决；（2）对角线竞争——单 kernel 内多 block 并发写入 $A[k][k]$ 产生未定义行为，拆分为 division+set_diag 两个 kernel 后消除。

5.4 性能测试结果与加速比

在 NVIDIA Tesla T4 GPU 上的性能测试结果如表 2 和图 ?? 所示。

表 2: CUDA 高斯消去性能测试结果

矩阵规模 N	CPU Serial (ms)	CUDA GPU (ms)	加速比
512	29.76	11.44	2.60×
1024	403.89	26.56	15.21×
2048	2832.34	142.34	19.90×

5.5 加速比分析

规模扩展性： GPU 加速比随矩阵规模增大而显著提升。 $N=512$ 时仅 2.60x，因为矩阵太小（仅 1 MB 数据量），3N 次 `cudaDeviceSynchronize` 的累积调用开销（约 30-150 ms）在总时间中占主导。 $N=2048$ 时计算量增至 $O(N^3)$ 的支配地位，GPU 的 5120 CUDA Cores 大规模并行优势充分释放，加速比达到 19.90x。

与 Lab 系列历史数据对比： 表 3 列出了本课程四次实验在同一问题（普通高斯消去）上的最高加速比。

表 3: 高斯消去四次实验最高加速比对比（ $N=2048$ ）

实验	并行模型	平台	最高加速比
Lab2 SIMD	NEON 向量化	鲲鹏 920 (8 核)	2.07×
Lab3 多线程	OpenMP+NEON (8 线程)	鲲鹏 920 (8 核)	9.64×
Lab4 MPI	MPI (4 进程)	鲲鹏 920 集群	2.43×
Lab5 GPU	CUDA	Tesla T4 (5120 cores)	19.90×

与指导书数据对比： 指导书给出的 NVIDIA P100 参考数据为 $N=1024$ 时 94x 加速（2157 ms $\rightarrow$ 22.97 ms）。本实验 T4 的结果为 15.21x（403.89 ms $\rightarrow$ 26.56 ms）。加速比绝对值差异主要由 CPU 基线不同导致——本实验的 CPU 串行版本采用了严格对角占优矩阵初始化和 O2 优化的 nvcc 编译，串行执行时间（404 ms）远低于指导书（2158 ms，可能是 O0 编译或较弱的 CPU）。GPU 侧的绝对执行时间（26.56 ms vs 22.97 ms）表明 T4 和 P100 在计算能力上接近。

6 总结

本次 GPU 编程实验通过 HIP 课程系统学习和 CUDA 实战编程，掌握了 GPU 并行编程的核心概念和方法：

- 线程与内存模型：** GPU 通过 Grid/Block/Thread 三级层次组织海量线程，wavefront（AMD）和 warp（NVIDIA）是最基本的执行单元。内存层次从 Global Memory（400+ cycles）到 Shared Memory（ ~ 20 cycles）的延迟差异巨大，合理利用 Shared Memory 是性能优化的核心。

2. **合并访问**：同一 wavefront 内的连续线程访问连续地址时，GPU 可合并为单次事务；跨步访问则需要每线程独立事务，带宽利用率骤降到 $1/32$ 。
3. **原子操作与竞争**：atomicAdd 解决了 race condition，但竞争会严重串行化执行。两级归约（LDS 内归约 $\rightarrow$ global 合并）将 global atomics 从 $O(N)$ 降至 $O(\text{gridDim})$ ，获得数量级的性能提升（Lab3 中达 201x）。
4. **树形归约与 Warp Shuffle**： $O(\log_2 N)$ 步归约 + Warp Shuffle 消除 bank conflict, Multi-Element per Thread 提升算术强度。
5. **CUDA 高斯消去**：在 Tesla T4 上 $N=2048$ 时达到 19.90x 加速比，是 Lab3 OpenMP+NEON (9.64x) 的 2 倍，展现了 GPU 在计算密集型任务上的巨大优势。

附录：Git 项目链接

本实验完整代码已托管至 Git 平台：

<https://github.com/NKU025/Parallel-Gaussian-Elimination>

参考文献

- [1] AMD Corporation. *HIP Programming Guide v4.5* [EB/OL]. <https://rocm.docs.amd.com/projects/HIP/en/latest/>
- [2] NVIDIA Corporation. *CUDA C++ Programming Guide* [EB/OL]. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [3] Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP* [M]. McGraw-Hill, 2004.

Appendix A Lab 1: Vector Addition 填空答案

Exercise 1: Thread Index Calculation

公式: $\text{Global Index} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$

blockIdx.x	blockDim.x	threadIdx.x	Global Index
0	256	100	100
2	128	50	306
5	64	0	320

Exercise 2: GPU Specifications

Radeon 8060S Graphics, 20 CUs, Wavefront Size = 32, Max Threads/Block = 1024, Max Shared Memory = 64 KB。最大同时执行线程数: $20 \times 64 \times 32 = 40960$ 。

Exercise 3: Launch Configuration (N = 1000)

Block Size	Grid Size	Total Threads	Idle Threads
64	16	1024	24
128	8	1024	24
256	4	1024	24
512	2	1024	24

推荐 BS=256, 平衡 occupancy 与资源

使用。

Exercise 4: Performance

BS=512 最优 (0.0179 ms); Vector Add 是 memory-bound——3 次访存 vs 1 次计算, 瓶颈在带宽。

Exercise 5: Wavefront

BS=100 wave32: 4 wavefronts, 28 wasted。BS=100 wave64: 2 wavefronts, 28 wasted。分支发散: 同一 warp 内分支串行执行。

Exercise 6: Occupancy

BS	WF/Block	Active WF/CU	Notes
32	1	64	max threads 限制
64	2	64	同上
128	4	64	同上
256	8	64	同上

Exercise 7: Sub-Wavefront

BS=8 (0.0694 ms) 比 BS=128 (0.0184 ms) 慢 3.8 倍。MI100 (wave64) 下 BS < 64 全为子波前。

Challenge

Multi-element: $\text{stride} = \text{blockDim.x} \times \text{gridDim.x}$ 。一次 launch 处理任意 N，增大算术强度。

Appendix B Lab 2: Matrix Transpose 填空答案

Exercise 1: 2D Indexing

blockIdx	threadIdx	Global (idx, idy)
(0,0)	(5,10)	(5,10)
(1,0)	(0,0)	(32,0)
(2,3)	(15,20)	(79,116)

rows=100, cols=200: $\lceil 200/32 \rceil \times \lceil 100/32 \rceil = 7 \times 4 =$

28 blocks。

Exercise 2: Index Transformation

4×6 matrix:	Thread	Input Index	Output Index
	(1,0)	1	4
	(0,1)	6	1
	(3,2)	15	14

写公式用 $\text{idx} \times \text{rows}$: 转置后 output 是

$\text{cols} \times \text{rows}$, 每行有 rows 个元素。

Exercise 3: Performance

Test	Elements	Time (ms)
16×16	256	0.0079
128×32	4,096	0.0091
1×1024	1,024	0.0069
1001×2001	2,003,001	0.1116

元素数 489 倍, 时间仅 12.3 倍——小测试由 launch over-

head 主导, 大测试跨步写入限制带宽。

Exercise 4: Coalescing

Q1: Yes, 合并访问。Q2: Stride = rows (1024)。Q3: 32 次事务 (vs 合并访问仅需 1 次)。

Exercise 5: Tiled Transpose

Q1: 加 1 列避免 shared memory bank conflict。Q2: 转置后 block 坐标互换。Q3: $32 \times 33 \times 4 = 4224$ bytes。

Challenge

去掉 +1 padding 导致 32-way bank conflict。Rectangular tile: 32×8 减少 LDS 用量。Double-buffering 隐藏 global memory 延迟。

Appendix C Lab 3: Histogram 填空答案

Exercise 1: Bad Implementation

Q1: 每个 element 被读 num_bins 次。Q2: $10\text{M} \times 256 \times 4 = 10.24\text{ GB}$ 。Q3: 重复读取浪费带宽。

Exercise 2: Optimization

Q1: LDS ~ 20 cycles vs global ~ 400 cycles。Q2: gridDim 次。Q3: 一次 launch 处理任意 N，增大算术强度隐藏延迟。

Exercise 3: Performance

Implementation	Test 2 (ms)	Test 4 (ms)
Serial CPU	2.4446	24.5800
Naive GPU	5.6085	360.5926
Optimized GPU	0.1865	1.7868

Optimized vs Serial: $13.8\times$, vs Naive: $201.8\times$ 。

Exercise 4: Memory Efficiency

Reduction = num_bins = $256\times$ 。

Exercise 5: Contention

All-same(0): 0.2706 ms (最慢，全部竞争 bin 0)，比 Uniform 慢 45%。

Challenge

Warp-level histogram: 每 warp 私有 register histogram, __shfl_down 归约。Competition data: Gaussian 0.1865 ms。num_bins > 1024 超出 LDS 限制。

Appendix D Lab 4: Parallel Reduction 填空答案

Exercise 1: Tree Reduction

Q1: $\log_2(256) = 8$ 步。Q2: gridDim 个 partial sum。Q3: 保证前一步写结果对下一步可见。

Exercise 2: Benchmark (N=1M, BS=256)

Strategy	Time (ms)	Speedup
Naive Atomic	0.0312	1.00×
Tree (Shared Memory)	0.0250	1.25×
Tree + Warp Shuffle	0.0204	1.53×
Multi-Element + Tree + Shuffle	0.0129	2.41×

算法效率 > occupancy。

Exercise 3: Trade-offs

BS=64: 15625 atomicAdds。BS=1024: 977 atomicAdds。更少 atomic = 更少竞争。

Exercise 4: Algorithm Analysis

Q1: 一半线程空闲。Q2: Warp 内 SIMT 锁步执行, __shfl_down 隐式同步。Q3: 算术强度增大, 隐藏内存延迟。

Challenge

Tree reduction: $\log_2(\text{blockDim})$ 步 LDS reduction。Warp shuffle: 最后 warpSize 步替代 sync。Multi-pass 用于 $N \gg \text{gridDim} \times \text{blockDim}$ 。

Appendix E Lab 5: Monte Carlo Integration 填空答案

Exercise 1: Algorithm

Q1: 每线程独立计算 $(b-a) \times y_i/n$ 。Q2: $O(N/P + \text{atomic contention})$ 。Q3: Shared memory tree reduction, global atomics 从 N 降为 gridDim。

Exercise 2: Manual Calculation

Sum of y = 23.7507, Result = $0.25 \times 23.7507 = 5.937675$ 。

Exercise 3: Scalability

Test	N	Time (s)
4	100,000	0.094
7	10,000,000	0.975
8	100,000,000	8.458

N 增 100 $\times$, 时间增 10.4 $\times$ 。大 N 接近线性但受 atomic contention

限制。

Exercise 4: Bottleneck

Q1: 0.99% 效率 (100M cycles 用于 atomic)。Q2: 降为 N/256 次 global atomics。Q3: 3907 次。

Exercise 5: Precision

Q1: double = 15-17 位。Q2: float 在大累加时截断小值。

Exercise 6: Optimization

Q1: 256 次 global atomics。Q2: 一次 launch 处理任意 N。Q3: partial sum 已汇总到 thread 0。

Challenge

Monte Carlo π : $\pi \approx 4 \times (\text{inside}/\text{total})$ 。Error $\propto 1/\sqrt{N}$ 。

Appendix F Lab 6: K-Means Clustering 填空答案

Exercise 1: Algorithm

Q1: $O(k) = O(100)$ 。Q2: LDS atomics (20ns) 远快于 global (400ns), 减少 contention。Q3: 保持 prev centroid, 不参与当前迭代更新。

Exercise 2: Performance

$N=50,000$, $k=50$, $\text{max_iter}=100$, $\text{Time} = 0.118\text{s}$ 。主导计算: Assignment step ($O(N \times k) = 2.5\text{M}$ 距离计算)。

Exercise 3: Compute Intensity

总距离计算: 2,500,000 次。Shared memory atomics: 150,000 次。Global atomics: 29,400 次。

Exercise 4: Memory Optimization

Q1: constant___ memory broadcast。Q2: $k=1000$ 时 instruction count 增长 $20\times$, 仍适合 constant cache。

Exercise 5: Convergence

Q1: 10^{-8} 。Q2: 节省计算资源, 避免无效迭代。Q3: 消除 hipMalloc/hipFree 开销。

Challenge

Dynamic shared memory: extern___ shared___。3D: 加 z 坐标, LDS 需求增 50%。K-Means++ 用 prefix sum + weighted sampling。Mini-batch: 每轮 10% 样本, 代价是更多迭代轮数。